Velarix: A Deterministic Truth Maintenance System for Verifiable AI Reasoning

Velarix Engineering
velarix.dev

April 2026

Abstract

Velarix is a high-performance Truth Maintenance System (TMS) and belief revision engine implemented in Go. Unlike probabilistic Large Language Models (LLMs) that suffer from logical volatility, Velarix offloads stateful reasoning to a deterministic Directed Acyclic Graph (DAG). By enforcing non-monotonic logic and formal causal justifications, the system provides a mathematically verifiable trace of an agent's reasoning. This document specifies the architecture, core data structures, and truth propagation algorithms that enable Velarix to scale to 80,000+ facts while maintaining absolute logical integrity.

1 System Architecture

The Velarix kernel is built on a reactive, concurrency-safe engine designed for low-latency state transitions.

1.1 Concurrency and State Management

The engine utilizes a global `sync.RWMutex` to ensure atomic belief updates. The state is represented as a memory-resident DAG of `Fact` and `JustificationSet` entities. This architecture prioritizes read-heavy workloads where multiple AI agents or audit processes query the truth-state simultaneously.

1.2 Persistence Layer

Velarix employs a pluggable storage interface (`store/interfaces.go`), supporting:

- **BadgerDB:** For local-first, high-throughput key-value persistence.
- **PostgreSQL:** For multi-tenant, enterprise-grade relational storage.
- **Redis:** For ephemeral, high-speed session caching.

2 Core Data Structures

The system's logic is defined by three primary symbolic structures:

2.1 Fact (Node)

A `Fact` represents an atomic belief. It contains:

- **ID**: A unique URN identifier.

- **Payload**: A schema-validated JSON-B object containing the belief content.
- **Status**: A float64 value (0.0 to 1.0) representing the derived validity.
- **AssertionKind**: Classification into `Empirical`, `Hypothetical`, or `Uncertain` to control contradiction-scope.

2.2 JustificationSet (Arc Set)

Velarix uses an OR-of-ANDs model for justifications. A `Fact` is considered *Valid* if at least one of its `JustificationSets` is fully satisfied. Each set defines:

$$J = \{P_1, P_2, \ldots, P_n\} \cup \{\neg N_1, \neg N_2, \ldots, \neg N_m\} \tag{1}$$

Where P represents positive dependencies and N represents negated (inhibitory) dependencies.

3 Logic & Belief Revision

Velarix implements a non-monotonic labeling algorithm for truth propagation.

3.1 Reactive Truth Propagation

When a root fact is modified or a new justification is added, the engine executes a `propagate` routine. This routine uses a work-queue to update the `DerivedStatus` of all downstream descendants. The system enforces a *Confidence Threshold* (typically 0.60); any fact falling below this threshold is treated as `Invalid`, triggering a recursive retraction across the graph.

3.2 Dominator Trees for Efficient Pruning

To handle deep dependency chains without $O(N^2)$ traversal, Velarix computes a **Dominator Tree**. This allows the engine to perform $O(1)$ pruning during truth queries: if a fact's Immediate Dominator (`IDom`) is invalidated, the fact is automatically considered invalid without requiring a full graph walk.

4 Technical Bottlenecks

The computational complexity of Velarix is driven by memory-resident graph operations.

4.1 Memory Complexity

The engine maintains a `MaxFactsPerSession` limit of 80,000 nodes. Due to the high connectivity of causal graphs (Forward and Reverse indices) and the requirement for in-memory dominator recomputation, high-depth simulations (e.g., counterfactual impact analysis on 50k+ nodes) require **32GB+ of RAM** to avoid swap-latency and OOM (Out-of-Memory) failures during mutation spikes.

5 Roadmap

- **LATTICE Integration:** Transitioning to transition-based transducers for 98%+ extraction accuracy.
- **Vector Symbolic Architectures (VSA):** Implementing HRR (Holographic Reduced Representations) for fuzzy entity grounding within the Go kernel.

- **Formal Verification:** Moving toward a TLA+ specification for the core propagation algorithm.